

buybuy闲置物品信息交流平台

buybuy是一个基于flask架构的类电商平台，其开发目的是为了开发出一个面向高校学生，由高校学生自主管理的交流闲置物品信息的平台。此项目亦可经过修改后用于其他方面。项目地址链接：

<https://gitlab.com/cs10102302-2020-tj/adventure-of-bug-coders/buybuy>

目录

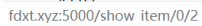
- 项目简介
- 使用说明
 - 功能说明
 - 管理员说明
 - 其他说明
- 安装部署
 - 依赖安装
 - 运行
 - 远程部署
- 开发与维护
 - 注意事项与说明
 - 项目目录结构
- 维护者
- License
- 更新日志

项目简介

本项目是一个面向高校学生、用于交流闲置物品信息、由高校学生自主研发、自主管理的非营利性平台。“简单易用，方便快捷”是我们应用开发的主要思路。我们将商品展示放在了更为重要的位置，而将交易部分简化为同学之间的线下交易，大大的简化了交易的过程。相对于物品种类较为繁杂的闲鱼，对于大学生的闲杂物品，我们的应用可以做到更好的搜索，分类，展示，浏览等。项目也可以对数据模型，前端显示，后端处理等方面进行修改，用于其他用途。

该项目目前主要具有的一些功能有：

- 以下是项目运行的一些截图演示:



用户名

密码



登录

使用说明

功能说明

1. 浏览、搜索、收藏商品，查看商品信息

- 在商品的主页，展示所有审核通过的商品信息
- 在整个项目下的各个页面中，页面头部均具有一个“Search”的搜索按钮，可以点击，输入信息，对商品进行搜索
- 点击主页加载出的商品图片，可以查看该商品的详细信息
- 在查看商品详细信息的页面，可以通过添加收藏按钮对商品进行收藏

2. 查看、删除商品浏览记录

- 每当用户查看了某个商品的详细信息，系统将会自动保存对应的浏览记录
- 用户可以通过个人中心中“浏览记录”，来查看自己已经访问过的商品
- 在“浏览记录”页面，用户可以通过点击删除浏览记录按钮，来对已经存在的浏览记录进行删除

3. 添加、删除商品，更新商品信息

- 经过管理员审核通过的用户可以通过个人中心的“添加商品”，来添加想要出售的商品，并且，通过个人中心中“我的商品”，可以查看到自己添加过的所有商品（按照是否被管理员审核通过区分开）
- 通过个人中心中“我的商品”，商家可以删除掉已经审核过的/未审核过的商品信息
- 通过个人中心中“我的商品”，商家可以修改已经审核过的/未审核的商品信息，并重新提交给管理员进行审核

4. 注册，登录，注销，个人信息维护功能

- 用户点击页面头部的注册按钮，给出用户名、密码以及邮箱来注册自己在本系统中的账号
- 在注册账号后，可以通过主页头部的登录按钮登录账号，并且正常使用系统的各个功能
- 在登录了账号之后，用户可以通过点击主页头部的注销按钮，来退出当前登录的账号
- 此外，在登录页面，用户可以通过“找回密码”来使用邮箱接受验证邮件，并重置自己的密码
- 用户还可以通过个人中心的“修改昵称”、“修改头像”、“修改密码”来分别对应的修改自己的用户名，头像以及密码，进行个人信息的维护

5. 支持买卖双方进行留言

- 在查看商品详细信息的页面，用户可以通过点击给卖家留言的按钮，来对卖家进行留言
- 买卖双方都可以通过个人中心中“消息列表”的方式，查看自己和其他用户之间的聊天记录

管理员说明

本系统中设置有管理员，进行用户身份认证的审核，用户上传商品的审核，以及封禁账号的功能。在项目中，管理员初始密码存储在config目录下的admin文件中，采用sha-256的存储形式，加盐是'buybuy'，初始的管理员账号为admin1，密码为password1。

1. 身份认证的审核

- 用户可以通过个人中心中“身份认证”，来上传自己的学生证照片，进行卖家的身份认证
- 管理员通过登录管理员账号，来审核所有上传学生证照片的用户，选择通过/不通过，如果不通过可以给出有关理由

2. 上传商品的审核

- 已经认证通过的卖家，可以通过添加商品来上传商品的信息，并提交给管理员审核
- 管理员通过登录管理员账号，来审核所有卖家上传的商品，并选择给出通过/不通过，如果不通过可以给出有关理由

3. 封禁账号

- 管理员通过登录管理员账号，来输入用户名，对对应的用户进行封号/解封的处理
- 被封禁的账号在登录的时候会提示账号被封禁，请联系管理员的信息

其他说明

1. 在本系统的各个页面的头部，均有“Search”按钮提供对商品的搜索
2. 在本系统的各个页面的头部，有一个用户的小人图标，鼠标放置在上面之后，提供一个到个人中心的跳转
3. 在本系统的各个页面的头部，提供了该页面到其他各个页面的跳转按钮
4. 本系统前端的页面采用 Bootstrap 进行编写，对移动端有比较好的兼容性，在移动端的页面，系统给出了menu按钮，提供了该页面到各个页面的跳转按钮

安装部署

依赖安装

该项目基于python3的flask架构，因此进行部署运行需要使用者电脑上具有python3环境。建议python具有3.7以上的版本。

首先，在该项目的目录下（以可以看到文件 app.py为准）打开命令行工具，或者将命令行工具切换到该目录下，安装依赖的python包。

为了避免包命名等冲突带来的问题，我们推荐在python的虚拟环境中部署该项目，虚拟环境的使用可以参考[这篇文章](#)。

您可以通过直接运行命令

```
python app.py
```

来逐渐查看缺少的包并且自行安装。

但是我们推荐直接运行命令

```
pip install -r requirements.txt
```

直接快速安装依赖包。

如果运行此命令时出现问题，那么请考虑是否出现了包重复等问题，可尝试在虚拟环境中运行或者删除已安装的包。

运行

在依赖安装完成后，可以在该目录下运行命令

```
python app.py
```

来运行此项目。项目正常启动后，可以通过访问本地 <http://127.0.0.1:5000/> 进行查看。

远程部署

这里介绍如何将项目布置到Linux操作系统的主机上。

如果想将此项目部署到远程云服务器上，首先需要将app.py中的

```
app.run(debug=True)
```

修改为

```
app.run(host="0.0.0.0",port=5000)
```

方便其他主机进行访问，其中端口号可以自行指定。

修改完成后，可以在项目目录下运行命令

```
nohup python app.py &
```

来将该项目进行后端挂起，项目产生的日志将会被保存在项目目录下的nohup.out文件中。

如果想要终止项目，可以考虑使用命令

```
ps aux|grep python
```

查看后台正在运行的程序，并且使用命令

```
kill pid (项目的pid编号)
```

的方式结束程序的运行。

部署完成后，其他主机可以通过该服务器的ip地址或者域名 + 端口号的方式对项目进行访问。

开发与维护

注意事项与说明

1. 该项目使用的是python的flask架构，采用的是典型的MVC结构，数据库采用的是SQLite。
2. 对于新页面的添加，首先需要去相应的view文件中实现路由函数，增加业务逻辑，之后需要去templates中添加相应的前端目录即可。
3. 在后期实际维护中，对数据模型的修改往往会导致与之前数据库的不兼容，目前最好的办法就是自行编写脚本，将原数据库中的内容以自定方式导出，根据新的数据模型进行相应修改后，再导入新数据库中，记得对数据库进行定期备份，防止数据丢失。

项目目录结构

项目的目录结构如下（部分）：

- └─account_manage //负责账号管理部分的数据模型与路由实现
- └─chat_manage //负责聊天部分的数据模型与路由实现
- └─config //一些配置文件，包括数据库等存储等
- └─db_manage //负责数据库部分的初始化等相关操作
- └─READMEIMG //储存readme文件中要显示的图片
- └─static //存储静态资源，包括商品图片等
 - └─assets
 - └─css
 - └─fonts
 - └─head_images //头像图片
 - └─identity_images //身份认证图片
 - └─img
 - └─js
 - └─libs
 - └─store_images //商品图片
- └─store_manage //负责商品信息的数据模型与路由实现
- └─templates //前端的显示模板
- └─test //单元测试等相关内容

维护者

该项目目前由同济大学计算机科学与技术系Adventure of Bug Coders小组开发维护。

如有其他问题或者想参与该项目请通过邮件联系 lby1570975210@gmail.com

License

- [MIT](#)

更新日志